DataCopilot – Module 2: Research and Solution Design

Nov 6, 2026 · @Mritunjay Sharma

Introduction

This document covers Module 2 of the DataCopilot capstone: the requirements, the research behind the design choices, the detailed solution design, and the plan for building it. DataCopilot lets users ask questions about CSV/Excel files or SQL databases in English or Hinglish. An LLM writes SQL, a validator checks it, and the database computes the answer, which is returned as a chart, a plain-language explanation, and the SQL used.

The design decisions here build on the approved Module 1 proposal and set the foundation for data preparation in Module 3.

1. Requirements Analysis

1.1 Business requirements

ID	Requirement
BR-1	Let non-technical users get answers from their own data without writing SQL or code.
BR-2	Reduce the risk of decisions based on incorrect AI-generated numbers.
BR-3	Make every answer verifiable by showing how it was produced.
BR-4	Run at zero licensing cost, using free and open-source tools.
BR-5	Support Indian users who mix Hindi and English (Hinglish).

1.2 Functional requirements

ID	Requirement	Priority
FR-1	Upload CSV or Excel files and load them into DuckDB.	Must
FR-2	Connect to SQLite and PostgreSQL databases in read-only mode.	Should
FR-3	Profile each dataset: tables, columns, types, row counts, missing values.	Must
FR-4	Show automatic insights: summary statistics, top correlations, anomalies.	Should
FR-5	Accept questions in English and Hinglish.	Must / Should
FR-6	Classify each question as answerable or unanswerable from the schema.	Must
FR-7	Generate SQL from the question and schema using an LLM.	Must
FR-8	Validate SQL syntax, schema references, and read-only safety before execution.	Must
FR-9	Retry generation once with the validator's error message if validation fails.	Should
FR-10	Execute validated SQL and return results.	Must
FR-11	Select a suitable chart type and generate a plain-language answer.	Must
FR-12	Display the SQL used and a verification status with every answer.	Must
FR-13	Require user login before uploading data.	Must
FR-14	Log every question, generated SQL, validation result, and response time.	Must

1.3 Non-functional requirements

ID	Category	Requirement
NFR-1	Performance	Median response time of 10 seconds or less on the cloud deployment.
NFR-2	Reliability	Validated pipeline produces a statistically significant reduction in wrong answers versus the baseline.
NFR-3	Security	No generated query can modify data; 100% of unsafe test queries blocked.
NFR-4	Privacy	Uploaded files are stored temporarily, are private to each user, and are deleted after the session.
NFR-5	Usability	A first-time user can upload a file and get an answer in under 2 minutes.
NFR-6	Maintainability	Each pipeline step is an independent, unit-tested module.
NFR-7	Portability	The application runs in a single Docker image.
NFR-8	Cost	Runs on cloud free tiers with a budget alert in place.

1.4 User requirements

User	Needs	Example question
Small-business owner	Quick totals and trends from sales spreadsheets	Which product sold the most last month?
Student or researcher	Explore datasets without coding	Average marks by gender in this survey?
Data analyst	Fast answers to repetitive questions, with SQL to reuse	Top 5 customers by revenue in 2025?
Hinglish speaker	Ask naturally in a mixed language	Kis city mein sabse zyada orders aaye?

1.5 Assumptions and constraints

- Data is tabular with header rows; uploads are limited to about 100 MB.
- Questions are analytical (filters, aggregates, rankings, trends), not open-ended forecasts.
- Local experiments use 7–8B open-source models on an 8 GB GPU; the cloud demo uses a hosted LLM API or a small model.
- Single developer, eight-week timeline, no paid services.

1.6 Success criteria

Success criteria from the Module 1 proposal remain unchanged: a significant reduction in wrong answers (McNemar's test, $p < 0.05$), answerability classifier F1 ≥ 0.85, 100% of unsafe queries blocked, median response ≤ 10 seconds, and an average UAT rating of at least 4 out of 5.

2. Research Review

2.1 Existing solutions

Solution	Approach	Strengths	Gaps DataCopilot addresses
ChatGPT data analysis, Julius AI	LLM writes and runs Python on uploaded files	Flexible, polished interfaces	Generated code runs without schema validation; no refusal for unanswerable questions; paid tiers
Vanna (open source, MIT)	RAG over DDL, documentation and example SQL stored in a vector database	Works with any SQL database and with local LLMs via Ollama; auto-generates Plotly charts	Accuracy depends on the training data provided; no built-in answerability check or measured error reduction
PandasAI (open source)	LLM generates pandas code over dataframes	Simple for CSV analysis	Executes generated code; same validation gap

Solution	Approach	Strengths	Gaps DataCopilot addresses
Databricks Genie, Power BI Copilot	Enterprise natural-language querying on governed data	Strong metadata and governance	Requires enterprise platforms and licences

2.2 Academic research

Benchmarks. BIRD (NeurIPS 2023) contains 12,751 question–SQL pairs across 95 databases totalling 33.4 GB in 37 professional domains, and emphasises dirty values and external knowledge. In the original paper GPT-4 reached 54.89% execution accuracy against a human result of 92.96%. By June 2026, Google reported 80.04% on the BIRD single-model leaderboard with Gemini-SQL2, still about 13 points below human performance. Spider (2018) is an older cross-database benchmark that is now considered easier.

Self-correction. Methods such as DIN-SQL and self-debugging feed execution errors back to the model and regenerate SQL until it runs. These improve accuracy but assume every question is answerable.

Answerability and abstention. TriageSQL and DTE classify questions as answerable or unanswerable. TrustSQL (2024) is the closest work to DataCopilot: it requires a model to either produce SQL or abstain, and tests pipelines that combine separate answerability detection, SQL generation and error detection. Its authors found that no method consistently beat a naive baseline that abstains on every question under their penalty-based reliability score, showing that reliable abstention is still an open problem.

2.3 Industry best practices

- Use read-only database credentials and allow only SELECT statements for generated SQL.
- Pass schema, column types, and sample values to the LLM rather than raw data.
- Validate and, where possible, dry-run SQL before execution; cap row limits and timeouts.
- Show users the generated SQL so results can be audited.
- Log questions and queries for monitoring and evaluation.

2.4 Technology evaluation

Need	Options considered	Choice	Reason
Query engine for files	pandas, SQLite, DuckDB	DuckDB	Reads CSV/Excel directly, fast analytical SQL, in-process, no server
SQL validation	Regex rules, database EXPLAIN, sqlglot	sqlglot	Parses SQL into a tree for exact statement and column checks; dialect-aware
LLM	Hosted APIs, Ollama local models	Ollama (Qwen2.5-Coder 7B, Llama 3.1 8B) plus optional API	Free and unlimited for experiments on an 8 GB GPU; API for the cloud demo
Answerability model	Rules, Logistic Regression, XGBoost, small transformer	Logistic Regression and XGBoost compared	Fast, explainable, trainable on a few hundred labelled questions
Backend	Flask, FastAPI	FastAPI	Async, automatic API docs, type validation
Frontend	React, Streamlit	Streamlit	Fast to build data apps in Python
Deployment	VM, serverless, container	Docker on a free-tier VM (AWS or GCP)	Portable and within budget

2.5 Key findings and recommendations

1. Even the best text-to-SQL systems are wrong on about one in five BIRD questions, so validation and transparency are necessary, not optional.
2. Existing open-source tools focus on generating SQL; few measure how much safeguards reduce wrong answers. This is DataCopilot's research contribution.
3. Abstention is hard, as TrustSQL shows. DataCopilot will report answerability results honestly, including false refusals.
4. Hinglish text-to-SQL is largely unexplored; a small Hinglish test set adds a novel angle.

5. A small local model will score well below the leaderboard. The evaluation should focus on the relative improvement from validation, not the absolute score.

3. Solution Design

3.1 System overview

DataCopilot is a layered web application. A Streamlit interface talks to a FastAPI backend, which runs an analysis pipeline over a data layer. The core design principle is separation of duties: the LLM only proposes SQL, deterministic code decides whether that SQL may run, and the database computes every number.

3.2 Component descriptions

Component	Module	Input	Output	Responsibility
Ingestion	`api/upload`	CSV/Excel file or DB connection string	Dataset ID, DuckDB table or read-only connection	Load data safely; enforce size limits
Profiler	`pipeline/profiler.py`	Dataset	Schema (tables, columns, types, sample values) and insights	Describe the data for the LLM and the user
Answerability classifier	`pipeline/answerability.py`	Question + schema	Answerable / unanswerable + confidence	Refuse questions the data cannot support
SQL generator	`pipeline/sql_generator.py`	Question + schema + rules	Candidate SQL	Prompt the LLM; extract SQL from the reply
Validator	`pipeline/validator.py`	Candidate SQL + schema	Pass or fail with reasons	Check syntax, tables, columns, SELECT-only; add row limit
Executor	`pipeline/executor.py`	Validated SQL	Result table	Run with timeout on DuckDB or SQLAlchemy
Explainer	`pipeline/explainer.py`	Question + result	Chart, answer text, SQL, status	Choose chart type; summarise the result

Component	Module	Input	Output	Responsibility
Logger	`api/logging`	Every request	Log records	Store question, SQL, validation result, latency

3.3 Request data flow

1. The user uploads a file or connects a database; the profiler stores the schema and shows insights.
2. The user asks a question in English or Hinglish.
3. The answerability classifier checks the question against the schema. If it is unanswerable, the user is told why and no SQL is generated.
4. The SQL generator prompts the LLM with the question, schema, sample values, and rules.
5. The validator checks the SQL. On failure, the generator retries once with the error message; on a second failure the user gets a clear error.
6. The executor runs the validated query with a row limit and timeout.
7. The explainer returns a chart, a plain-language answer, the SQL, and a verified status.
8. Every step is logged for monitoring and evaluation.

3.4 Security considerations

- **Authentication:** login required before upload; sessions expire.
- **Read-only execution:** validator allows only SELECT; external databases use read-only credentials.
- **Isolation:** each user's files are stored in a separate temporary location and deleted after the session.
- **Limits:** upload size cap, query timeout, and maximum returned rows.
- **Secrets:** API keys and database URLs are read from environment variables, never committed to Git.
- **Audit logging:** every generated query and validation decision is recorded.

3.5 Scalability considerations

The target is a single-user-at-a-time academic deployment on a free-tier server, so the design favours simplicity. DuckDB runs in-process and handles files of tens of megabytes quickly. The LLM is the slowest step; using a hosted API for the cloud demo removes GPU

requirements from the server. If usage grew, the stateless FastAPI service could run as several containers behind a load balancer, with uploads moved to object storage.

3.6 Deployment approach

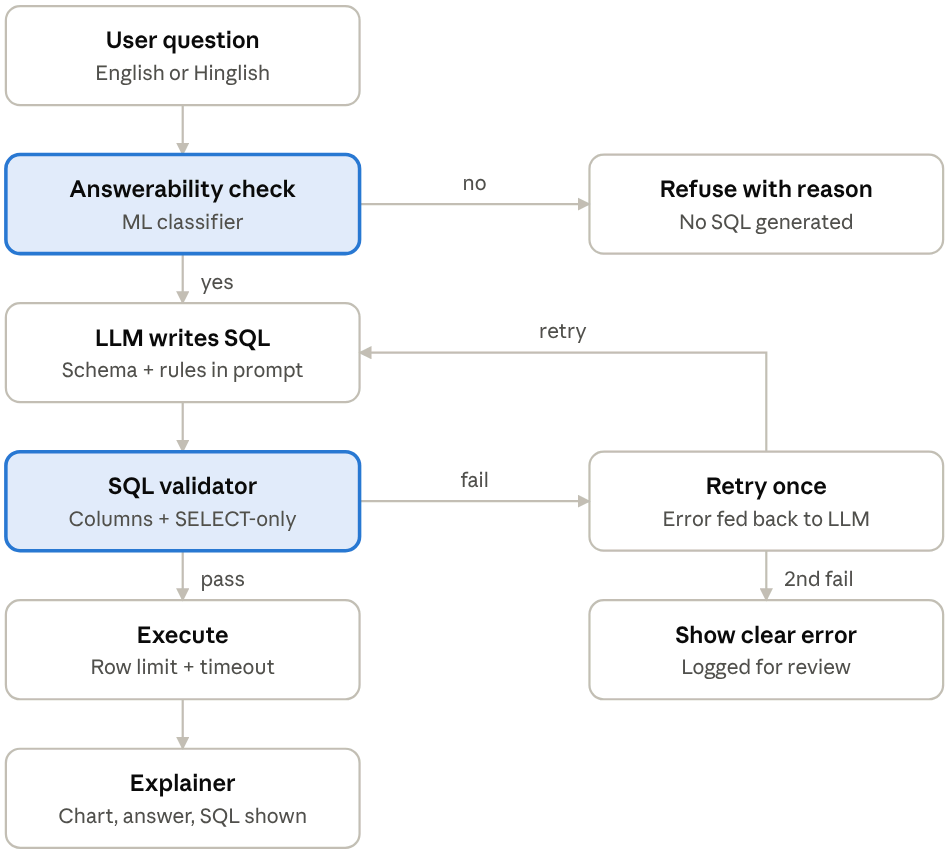
The whole application is packaged as one Docker image and deployed on a free-tier AWS EC2 or GCP Compute Engine VM. Configuration comes from environment variables. A budget alert is set before deployment. Development and benchmark experiments run locally on a laptop GPU with Ollama, so cloud usage stays minimal.

4. Detailed Architecture Diagrams

The high-level architecture from Module 1 still applies. The two diagrams below add detail on the request flow and the cloud deployment.

4.1 Request data flow

Two gates stop a wrong answer before the database runs anything

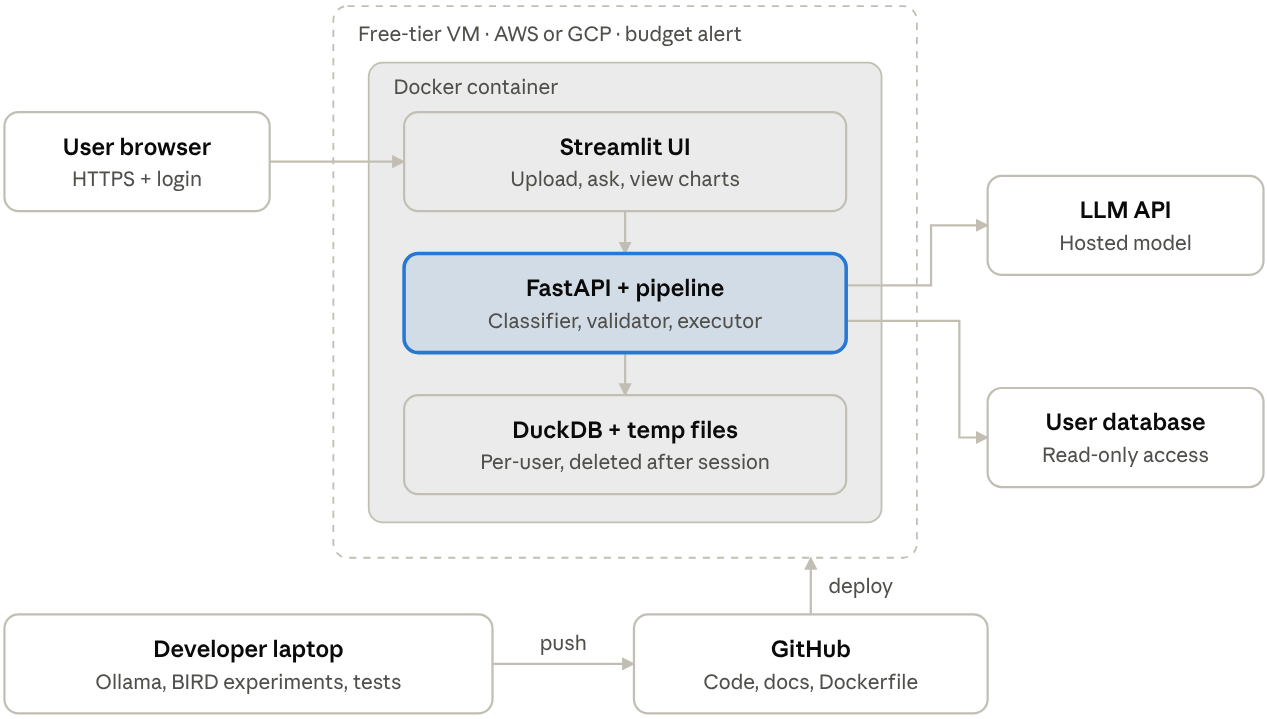


Request data flow · two validation gates

An answer reaches the user only after passing both gates. Unanswerable questions stop at the first gate, and SQL that fails validation gets one retry before the user sees a clear error.

4.2 Cloud deployment

One Docker container on a free-tier VM; heavy experiments stay on the laptop



Cloud deployment architecture

The deployed app needs no GPU: the server calls a hosted LLM API, while benchmark experiments with local models run on the developer laptop. User databases are reached with read-only credentials only.

5. Technology Stack Selection

The stack below is confirmed from the evaluation in section 2.4. Every component is free and open source, except the optional hosted LLM API for the cloud demo.

Layer	Technology	Justification
Language	Python 3.11	One language across data, ML, backend and UI
Data processing	Pandas, NumPy	Profiling and auto insights
Query engine	DuckDB	Fast analytical SQL directly on CSV/Excel, in-process

Layer	Technology	Justification
External databases	SQLAlchemy (SQLite, PostgreSQL)	One interface for multiple databases, read-only connections
SQL validation	sqlglot	Parse tree for exact statement, table and column checks
LLM (experiments)	Ollama with Qwen2.5-Coder 7B and Llama 3.1 8B	Free, local, fits an 8 GB GPU; enables model comparison
LLM (cloud demo)	Hosted LLM API (provider to be confirmed)	Free-tier servers cannot run local models
Answerability ML	scikit-learn (Logistic Regression), XGBoost	Small labelled dataset; explainable; fast inference
Backend	FastAPI, Uvicorn	Async API with automatic docs and validation
Frontend	Streamlit	Rapid data-app development in Python
Charts	Plotly	Interactive charts, easy type selection
Testing	Pytest, BIRD evaluation scripts	Unit tests and benchmark evaluation
Packaging and deployment	Docker, AWS EC2 or GCP Compute Engine (free tier)	Portable, low cost
Version control	Git, GitHub	Code, documents and progress history

6. Implementation and Project Execution Plan

6.1 Development phases

Module	Phase	Key activities	Depends on
2	Design + prototype	This document; basic prototype (CSV → DuckDB → LLM SQL → validator → answer); schema check in validator	Module 1 approval
3	Data preparation	Download BIRD dev set; build CSV, Hinglish and unanswerable question sets; EDA; label data for the classifier	Prototype for running questions
4	Model development	Train and compare answerability classifiers; run baseline vs validated pipeline on all question sets; statistical testing	Labelled question sets
5	Cloud deployment	Dockerfile; deploy to free-tier VM; login, logging, budget alert	Working pipeline
6	Integration	Streamlit UI, charts, auto insights, PostgreSQL connection, end-to-end flow	Deployed backend
7	Testing and optimisation	Functional, performance, security and user acceptance testing; user guide; draft report	Integrated system
8	Final delivery	Final report, presentation, live demo, research paper draft	All previous phases

6.2 Resource requirements

- **Hardware:** laptop with RTX 4060 (8 GB VRAM) for local models and experiments; about 10 GB of disk space for the BIRD dev set and models.
- **Software:** all open source (see section 5).
- **Cloud:** one free-tier VM; optional hosted LLM API key.
- **People:** one developer; supervisor reviews; five or more UAT volunteers in Module 7.

6.3 Technical risks and mitigation

Project-level risks are listed in the Module 1 charter. Design-specific risks:

Risk	Mitigation
Validator passes SQL that is valid but answers the wrong question	Measured explicitly on BIRD; reported as a known limitation
LLM output contains text around the SQL	Strict prompt format plus extraction and parsing step
Hinglish questions reduce SQL accuracy	Measured separately; optional translation step tested if the gap is large
Answerability classifier over-refuses	Track false refusals; tune decision threshold on a validation set
Large BIRD databases slow experiments	Run overnight; cache results; use dev set only

6.4 Milestones and deadlines

Each module ends with its deliverables submitted on the portal and pushed to GitHub. Scope is locked at the end of this module: no new features after Module 2. If the schedule slips, optional features are dropped in this order: full BIRD evaluation, then PostgreSQL support, then Hinglish.

References

1. Li, J., et al. (2023). *Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs.* NeurIPS 2023. arxiv.org/abs/2305.03111
2. Lee, G., Chay, W., Cho, S., & Choi, E. (2024). *TrustSQL: Benchmarking Text-to-SQL Reliability with Penalty-Based Scoring.* arxiv.org/abs/2403.15879
3. Yu, T., et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.* EMNLP 2018.
4. Zhang, Y., et al. (2020). *Did You Ask a Good Question? A Cross-Domain Question Intention Classification Benchmark for Text-to-SQL (TriageSQL).*
5. Pourreza, M., & Rafiei, D. (2023). *DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction.* NeurIPS 2023.

6. Chen, X., et al. (2023). *Teaching Large Language Models to Self-Debug.*
7. AI Weekly (2026). *Google's Gemini-SQL2 Tops BIRD Text-to-SQL at 80.04%.* aiweekly.co
8. Vanna: open-source RAG framework for SQL generation. github.com/vanna-ai/vanna
9. BIRD benchmark leaderboard. bird-bench.github.io
10. sqlglot documentation. github.com/tobymao/sqlglot
11. DuckDB documentation. duckdb.org
12. Ollama. ollama.com
13. McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.